PLUGINS

Discover and install prebuilt plugins through marketplaces

Find and install plugins from marketplaces to extend Claude Code with new skills, agents, and capabilities.

Plugins extend Claude Code with skills, agents, hooks, and MCP servers. Plugin marketplaces are catalogs that help you discover and install these extensions without building them yourself.

Looking to create and distribute your own marketplace? See **Create and distribute a plugin marketplace**.

How marketplaces work

A marketplace is a catalog of plugins that someone else has created and shared. Using a marketplace is a two-step process:

1. **Add the marketplace**

   This registers the catalog with Claude Code so you can browse what's available. No plugins are installed yet.

2. **Install individual plugins**

   Browse the catalog and install the plugins you want.

Think of it like adding an app store: adding the store gives you access to browse its collection, but you still choose which apps to download individually.

Official Anthropic marketplace

The official Anthropic marketplace (`claude-plugins-official`) is automatically available when you

start Claude Code. Run `/plugin` and go to the **Discover** tab to browse what's available, or view the catalog at **claude.com/plugins**.

To install a plugin from the official marketplace, use `/plugin install <name>@claude-plugins-official`. For example, to install the GitHub integration:

```
/plugin install github@claude-plugins-official
```

If Claude Code reports that the plugin is not found in any marketplace, your marketplace is either missing or outdated. Run `/plugin marketplace update claude-plugins-official` to refresh it, or `/plugin marketplace add anthropics/claude-plugins-official` if you haven't added it before. Then retry the install.

> The official marketplace is curated by Anthropic, and inclusion is at Anthropic's discretion. The in-app submission forms add plugins to the **community marketplace**, not the official one. To distribute plugins independently, **create your own marketplace** and share it with users.

The official marketplace includes several categories of plugins:

Code intelligence

Code intelligence plugins enable Claude Code's built-in LSP tool, giving Claude the ability to jump to definitions, find references, and see type errors immediately after edits. These plugins configure **Language Server Protocol** connections, the same technology that powers VS Code's code intelligence.

These plugins require the language server binary to be installed on your system. If you already have a language server installed, Claude may prompt you to install the corresponding plugin when you open a project.

Language	Plugin	Binary required
C/C++	`clangd-lsp`	`clangd`
C#	`csharp-lsp`	`csharp-ls`
Go	`gopls-lsp`	`gopls`
Java	`jdtls-lsp`	`jdtls`
Kotlin	`kotlin-lsp`	`kotlin-language-server`

Language	Plugin	Binary required
Lua	`lua-lsp`	`lua-language-server`
PHP	`php-lsp`	`intelephense`
Python	`pyright-lsp`	`pyright-langserver`
Rust	`rust-analyzer-lsp`	`rust-analyzer`
Swift	`swift-lsp`	`sourcekit-lsp`
TypeScript	`typescript-lsp`	`typescript-language-server`

You can also **create your own LSP plugin** for other languages.

> If you see `Executable not found in $PATH` in the `/plugin` Errors tab after installing a plugin, install the required binary from the table above.

What Claude gains from code intelligence plugins

Once a code intelligence plugin is installed and its language server binary is available, Claude gains two capabilities:

- **Automatic diagnostics**: after every file edit Claude makes, the language server analyzes the changes and reports errors and warnings back automatically. Claude sees type errors, missing imports, and syntax issues without needing to run a compiler or linter. If Claude introduces an error, it notices and fixes the issue in the same turn. This requires no configuration beyond installing the plugin. You can see diagnostics inline by pressing **Ctrl+O** when the "diagnostics found" indicator appears.
- **Code navigation**: Claude can use the language server to jump to definitions, find references, get type info on hover, list symbols, find implementations, and trace call hierarchies. These operations give Claude more precise navigation than grep-based search, though availability may vary by language and environment.

If you run into issues, see **Code intelligence troubleshooting**.

External integrations

These plugins bundle pre-configured **MCP servers** so you can connect Claude to external services without manual setup:

- **Source control**: `github`, `gitlab`

- **Project management**: `atlassian` (Jira/Confluence), `asana`, `linear`, `notion`
- **Design**: `figma`
- **Infrastructure**: `vercel`, `firebase`, `supabase`
- **Communication**: `slack`
- **Monitoring**: `sentry`

Automatic security review

The `security-guidance` plugin reviews each change Claude makes for common vulnerabilities and instructs Claude to fix what it finds in the same session. See **Catch security issues as Claude writes code** for what it checks and how to add project-specific rules.

Development workflows

Plugins that add skills and agents for common development tasks:

- **commit-commands**: Git commit workflows including commit, push, and PR creation
- **pr-review-toolkit**: specialized agents for reviewing pull requests
- **agent-sdk-dev**: tools for building with the Claude Agent SDK
- **plugin-dev**: toolkit for creating your own plugins

Output styles

Customize how Claude responds:

- **explanatory-output-style**: educational insights about implementation choices
- **learning-output-style**: interactive learning mode for skill building

Community marketplace

The community marketplace at `anthropics/claude-plugins-community` hosts third-party plugins that have passed Anthropic's automated validation and safety screening. Each plugin is pinned to a specific commit SHA in the catalog. Unlike the official marketplace, you add it manually:

```
/plugin marketplace add anthropics/claude-plugins-community
```

Then install plugins from it using the `claude-community` marketplace name:

```
/plugin install <plugin-name>@claude-community
```

To submit your own plugin to the community marketplace, see **Submit your plugin to the community marketplace** in the create-plugins guide.

Try it: add the demo marketplace

Anthropic also maintains a **demo plugins marketplace** (`claude-code-plugins`) with example plugins that show what's possible with the plugin system. Unlike the official marketplace, you need to add this one manually.

1 Add the marketplace

From within Claude Code, run the `plugin marketplace add` command for the `anthropics/claude-code` marketplace:

```
/plugin marketplace add anthropics/claude-code
```

This downloads the marketplace catalog and makes its plugins available to you.

2 Browse available plugins

Run `/plugin` to open the plugin manager. This opens a tabbed interface with four tabs you can cycle through using **Tab**, or **Shift+Tab** to go backward:

- **Discover**: browse available plugins from all your marketplaces
- **Installed**: view and manage your installed plugins
- **Marketplaces**: add, remove, or update your added marketplaces
- **Errors**: view any plugin loading errors

Go to the **Discover** tab to see plugins from the marketplace you just added. When your administrator has allowlisted the marketplace via the `pluginSuggestionMarketplaces` managed setting, plugins marked as relevant to your current working directory are pinned at the top with a **suggested for this directory** label.

3 Install a plugin

Select a plugin to view its details. The details pane shows what the plugin contains and what it costs:

- A **Context cost** estimate so you can see how many tokens the plugin will add to your **context window** every turn (Claude Code v2.1.143 and later)
- The plugin's **Last updated** date (v2.1.144 and later)
- A **Will install** section listing the plugin's commands, agents, skills, hooks, and MCP and LSP servers, so you can review exactly what it adds before installing (v2.1.145 and later)

Choose an installation scope:

- **User scope**: install for yourself across all projects
- **Project scope**: install for all collaborators on this repository
- **Local scope**: install for yourself in this repository only

For example, select **commit-commands**, a plugin that adds git workflow skills, and install it to your user scope.

You can also install directly from the command line:

```
/plugin install commit-commands@claude-code-plugins
```

See **Configuration scopes** to learn more about scopes.

4 **Use your new plugin**

After installing, run `/reload-plugins` to activate the plugin. Plugin skills are namespaced by the plugin name, so **commit-commands** provides skills like `/commit-commands:commit`.

Try it out by making a change to a file and running:

```
/commit-commands:commit
```

This stages your changes, generates a commit message, and creates the commit.

Each plugin works differently. Check the plugin's details in the **Discover** tab to see the commands and skills it provides, or visit its homepage for usage guidance.

The rest of this guide covers all the ways you can add marketplaces, install plugins, and manage your configuration.

Add marketplaces

Use the `/plugin marketplace add` command to add marketplaces from different sources.

Shortcuts: You can use `/plugin market` instead of `/plugin marketplace`, and `rm` instead of `remove`.

- **GitHub repositories**: `owner/repo` format, for example `anthropics/claude-code`
- **Git URLs**: any git repository URL, including GitLab, Bitbucket, and self-hosted servers
- **Local paths**: directories or direct paths to `marketplace.json` files
- **Remote URLs**: direct URLs to hosted `marketplace.json` files

Add from GitHub

Add a GitHub repository that contains a `.claude-plugin/marketplace.json` file using the `owner/repo` format, where `owner` is the GitHub username or organization and `repo` is the repository name.

For example, `anthropics/claude-code` refers to the `claude-code` repository owned by `anthropics`:

```
/plugin marketplace add anthropics/claude-code
```

Add from other Git hosts

Add any git repository by providing the full URL. This works with any Git host, including GitLab, Bitbucket, and self-hosted servers. Include the `.git` suffix so Claude Code clones the repository rather than treating the URL as a direct link to a hosted `marketplace.json` file.

Using HTTPS:

```
/plugin marketplace add https://gitlab.com/company/plugins.git
```

Using SSH:

```
/plugin marketplace add git@gitlab.com:company/plugins.git
```

To add a specific branch or tag, append `#` followed by the ref:

```
/plugin marketplace add
https://gitlab.com/company/plugins.git#v1.0.0
```

Add from local paths

Add a local directory that contains a `.claude-plugin/marketplace.json` file:

```
/plugin marketplace add ./my-marketplace
```

You can also add a direct path to a `marketplace.json` file:

```
/plugin marketplace add ./path/to/marketplace.json
```

Add from remote URLs

Add a remote `marketplace.json` file via URL:

```
/plugin marketplace add https://example.com/marketplace.json
```

> URL-based marketplaces have some limitations compared to Git-based marketplaces. If you encounter "path not found" errors when installing plugins, see **Troubleshooting**.

Install plugins

Once you've added marketplaces, you can install plugins directly. The command installs to user scope by default:

```
/plugin install plugin-name@marketplace-name
```

To choose a different **installation scope**, use the interactive UI: run `/plugin`, go to the **Discover** tab, and press **Enter** on a plugin. You'll see options for:

- **User scope** (default): install for yourself across all projects
- **Project scope**: install for all collaborators on this repository, which adds the plugin to `.claude/settings.json`
- **Local scope**: install for yourself in this repository only, not shared with collaborators

You may also see plugins with **managed** scope. These are installed by administrators via **managed settings** and can't be modified.

> Make sure you trust a plugin before installing it. Anthropic doesn't control what MCP servers, files, or other software are included in plugins and can't verify that they work as intended. Check each plugin's homepage for more information.

Manage installed plugins

Run `/plugin` and go to the **Installed** tab to view, enable, disable, or uninstall your plugins. The list is grouped by scope and sorted so you see problems first: plugins with load errors or unresolved dependencies appear at the top, followed by your favorites, with disabled plugins folded behind a

collapsed header at the bottom.

From the list you can:

- press `f` to favorite or unfavorite the selected plugin
- type to filter by plugin name or description
- press Enter to open a plugin's detail view and enable, disable, or uninstall it

The detail view shows the components the plugin contributes: commands, skills, agents, hooks, MCP servers, and LSP servers. The same inventory is available from the command line with `claude plugin details`.

In Claude Code v2.1.187 and later, the Installed tab adds a **Not used recently** group for marketplace plugins you installed yourself but haven't invoked in at least two weeks, over a span of at least 10 sessions, and the detail view shows a **Last used** line for each plugin. Use these to find plugins that you no longer use but that are still adding startup and context cost, then disable or uninstall them.

Plugins that your organization manages or that you load with `--plugin-dir` are never listed as unused, and plugins that contribute an LSP server, theme, output style, monitor, or workflow are also never listed, since those deliver value without an invocation to track. The group and the **Last used** line are both hidden when your organization restricts marketplaces with `strictKnownMarketplaces`.

When you install a plugin that declares dependencies, the install output lists which dependencies were auto-installed alongside it.

You can also manage plugins with direct commands.

List installed plugins without opening the menu:

```
/plugin list
```

Pass `--enabled` or `--disabled` to show only plugins in that state.

Disable a plugin without uninstalling:

```
/plugin disable plugin-name@marketplace-name
```

Re-enable a disabled plugin:

```
/plugin enable plugin-name@marketplace-name
```

Completely remove a plugin:

```
/plugin uninstall plugin-name@marketplace-name
```

The `--scope` option lets you target a specific scope with CLI commands:

```
claude plugin install formatter@your-org --scope project
claude plugin uninstall formatter@your-org --scope project
```

Apply plugin changes without restarting

When you install, enable, or disable plugins during a session, run `/reload-plugins` to pick up all changes without restarting:

```
/reload-plugins
```

Claude Code reloads all active plugins and shows counts for plugins, skills, agents, hooks, plugin MCP servers, and plugin LSP servers.

Reloading has a token cost on the next request: newly loaded components announce themselves in content appended to the conversation, while the existing history still reads from the prompt cache. A plugin that provides MCP servers costs more when its tools aren't deferred by **tool search**: the change invalidates the cache and the next request re-reads the entire conversation. In that case `/reload-plugins` shows a warning and does not apply the reload; pass `--force` to apply anyway. See **enabling or disabling a plugin** for details.

Manage marketplaces

You can manage marketplaces through the interactive `/plugin` interface or with CLI commands.

Use the interactive interface

Run `/plugin` and go to the **Marketplaces** tab to:

- View all your added marketplaces with their sources and status
- Add new marketplaces
- Update marketplace listings to fetch the latest plugins
- Remove marketplaces you no longer need

Use CLI commands

You can also manage marketplaces with direct commands.

List all configured marketplaces:

```
/plugin marketplace list
```

Refresh plugin listings from a marketplace:

```
/plugin marketplace update marketplace-name
```

Remove a marketplace:

```
/plugin marketplace remove marketplace-name
```

⚠ Removing a marketplace will uninstall any plugins you installed from it.

Configure auto-updates

Claude Code can automatically update marketplaces and their installed plugins at startup. When auto-update is enabled for a marketplace, Claude Code refreshes the marketplace data and updates installed plugins to their latest versions. If any plugins were updated, you'll see a notification prompting you to run `/reload-plugins`.

Toggle auto-update for individual marketplaces through the UI:

1. Run `/plugin` to open the plugin manager
2. Select **Marketplaces**

3. Choose a marketplace from the list
4. Select **Enable auto-update** or **Disable auto-update**

Official Anthropic marketplaces have auto-update enabled by default. Third-party and local development marketplaces have auto-update disabled by default.

Administrators can also set `"autoUpdate": true` on each `extraKnownMarketplaces` entry in managed settings to enable auto-update for an organization marketplace without requiring each user to toggle it.

To disable all automatic updates entirely for both Claude Code and all plugins, set the `DISABLE_AUTOUPDATER` environment variable. See **Auto updates** for details.

To keep plugin auto-updates enabled while disabling Claude Code auto-updates, set `FORCE_AUTOUPDATE_PLUGINS=1` along with `DISABLE_AUTOUPDATER`:

```
export DISABLE_AUTOUPDATER=1
export FORCE_AUTOUPDATE_PLUGINS=1
```

This is useful when you want to manage Claude Code updates manually but still receive automatic plugin updates.

Configure team marketplaces

Team admins can set up automatic marketplace installation for projects by adding marketplace configuration to `.claude/settings.json`. When team members trust the repository folder, Claude Code prompts them to install these marketplaces and plugins.

Add `extraKnownMarketplaces` to your project's `.claude/settings.json`:

```
{
  "extraKnownMarketplaces": {
    "my-team-tools": {
      "source": {
        "source": "github",
        "repo": "your-org/claude-plugins"
      }
    }
  }
}
```

For full configuration options including `extraKnownMarketplaces` and `enabledPlugins`, see **Plugin settings**.

Security

Plugins and marketplaces are highly trusted components that can execute arbitrary code on your machine with your user privileges. Only install plugins and add marketplaces from sources you trust. Organizations can restrict which marketplaces users are allowed to add using **managed marketplace restrictions**.

Troubleshooting

/plugin command not recognized

If you see “unknown command” or the `/plugin` command doesn’t appear:

1. **Check your version**: run `claude --version` to see what’s installed.
2. **Update Claude Code**:
   - **Homebrew**: `brew upgrade claude-code`, or `brew upgrade claude-code@latest` if you installed that cask
   - **npm**: `npm install -g @anthropic-ai/claude-code@latest`
   - **Native installer**: re-run the install command from **Setup**
3. **Restart Claude Code**: after updating, restart your terminal and run `claude` again.

Common issues

- **Marketplace not loading**: verify the URL is accessible and that `.claude-plugin/marketplace.json` exists at the path
- **Plugin installation failures**: check that plugin source URLs are accessible and that repositories are public, or that you have access to them
- **Files not found after installation**: plugins are copied to a cache, so paths referencing files outside the plugin directory won't work
- **Plugin skills not appearing**: clear the cache with `rm -rf ~/.claude/plugins/cache`, restart Claude Code, and reinstall the plugin.

For detailed troubleshooting with solutions, see **Troubleshooting** in the marketplace guide. For debugging tools, see **Debugging and development tools**.

Code intelligence issues

- **Language server not starting**: verify the binary is installed and available in your `$PATH`. Check the `/plugin` Errors tab for details.
- **High memory usage**: language servers like `rust-analyzer` and `pyright` can consume significant memory on large projects. If you experience memory issues, disable the plugin with `/plugin disable <plugin-name>` and rely on Claude's built-in search tools instead.
- **False positive diagnostics in monorepos**: language servers may report unresolved import errors for internal packages if the workspace isn't configured correctly. These don't affect Claude's ability to edit code.

Next steps

- **Build your own plugins**: see **Plugins** to create skills, agents, and hooks
- **Create a marketplace**: see **Create a plugin marketplace** to distribute plugins to your team or community
- **Technical reference**: see **Plugins reference** for complete specifications